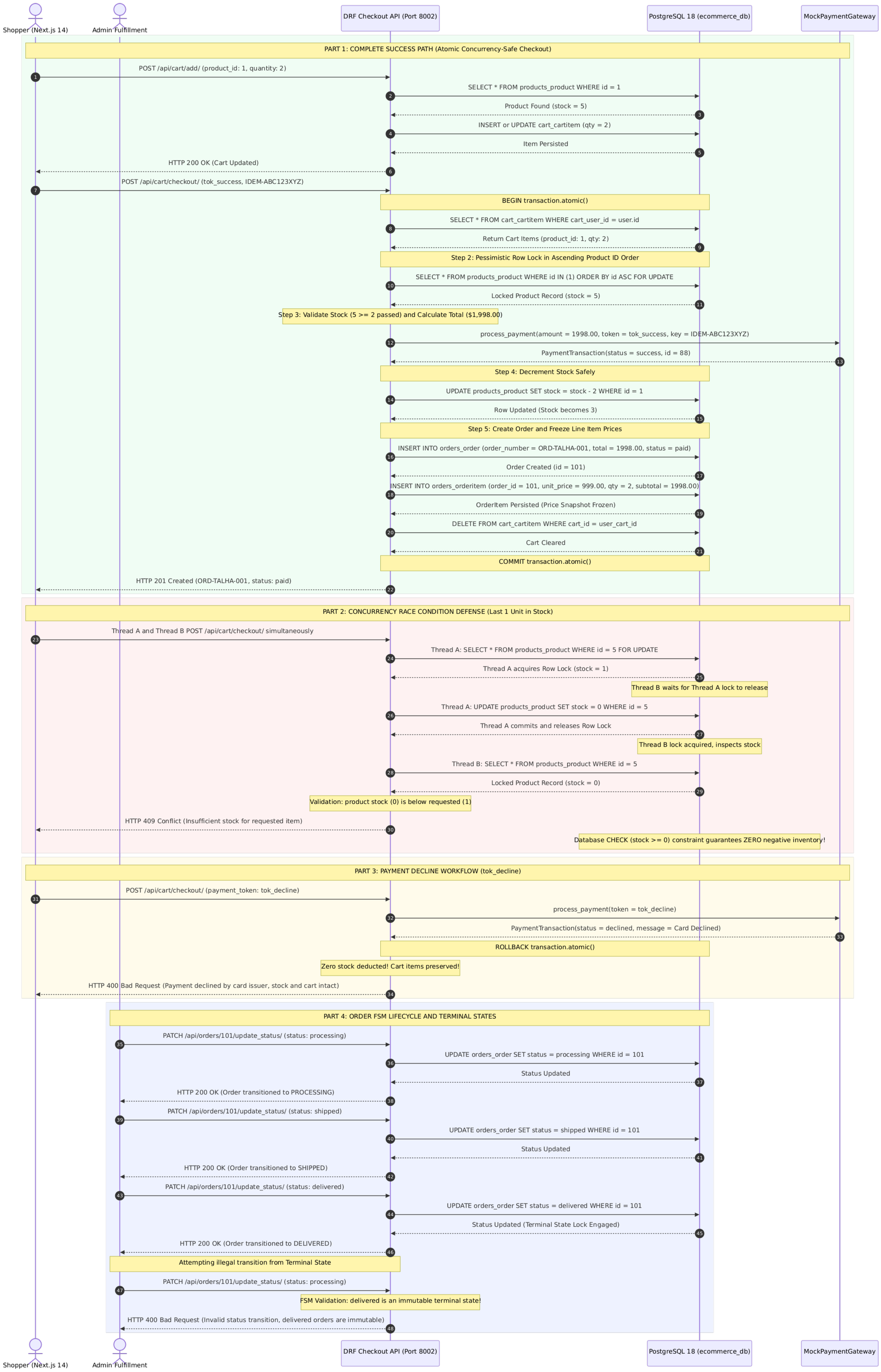

Shopper (Next.js 14)
Admin Fulfillment
DRF Checkout API (Port 8002)
PostgreSQL 18 (ecommerce_db)
MockPaymentGateway
PART 1: COMPLETE SUCCESS PATH (Atomic Concurrency-Safe Checkout)
POST /api/cart/add/ (product_id: 1, quantity: 2)
SELECT * FROM products_product WHERE id = 1
Product Found (stock = 5)
INSERT or UPDATE cart_cartitem (qty = 2)
Item Persisted
HTTP 200 OK (Cart Updated)
POST /api/cart/checkout/ (tok_success, IDEM-ABC123XYZ)
BEGIN transaction.atomic()
SELECT * FROM cart_cartitem WHERE cart_user_id = user.id
Return Cart Items (product_id: 1, qty: 2)
Step 2: Pessimistic Row Lock in Ascending Product ID Order
SELECT * FROM products_product WHERE id IN (1) ORDER BY id ASC FOR UPDATE
Locked Product Record (stock = 5)
Step 3: Validate Stock (5 >= 2 passed) and Calculate Total ($1,998.00)
process_payment(amount = 1998.00, token = tok_success, key = IDEM-ABC123XYZ)
PaymentTransaction(status = success, id = 88)
Step 4: Decrement Stock Safely
UPDATE products_product SET stock = stock - 2 WHERE id = 1
Row Updated (Stock becomes 3)
Step 5: Create Order and Freeze Line Item Prices
INSERT INTO orders_order (order_number = ORD-TALHA-001, total = 1998.00, status = paid)
Order Created (id = 101)
INSERT INTO orders_orderitem (order_id = 101, unit_price = 999.00, qty = 2, subtotal = 1998.00)
OrderItem Persisted (Price Snapshot Frozen)
DELETE FROM cart_cartitem WHERE cart_id = user_cart_id
Cart Cleared
COMMIT transaction.atomic()
HTTP 201 Created (ORD-TALHA-001, status: paid)
PART 2: CONCURRENCY RACE CONDITION DEFENSE (Last 1 Unit in Stock)
Thread A and Thread B POST /api/cart/checkout/ simultaneously
Thread A: SELECT * FROM products_product WHERE id = 5 FOR UPDATE
Thread A acquires Row Lock (stock = 1)
Thread B waits for Thread A lock to release
Thread A: UPDATE products_product SET stock = 0 WHERE id = 5
Thread A commits and releases Row Lock
Thread B lock acquired, inspects stock
Thread B: SELECT * FROM products_product WHERE id = 5
Locked Product Record (stock = 0)
Validation: product stock (0) is below requested (1)
HTTP 409 Conflict (Insufficient stock for requested item)
Database CHECK (stock >= 0) constraint guarantees ZERO negative inventory!
PART 3: PAYMENT DECLINE WORKFLOW (tok_decline)
POST /api/cart/checkout/ (payment_token: tok_decline)
process_payment(token = tok_decline)
PaymentTransaction(status = declined, message = Card Declined)
ROLLBACK transaction.atomic()
Zero stock deducted! Cart items preserved!
HTTP 400 Bad Request (Payment declined by card issuer, stock and cart intact)
PART 4: ORDER FSM LIFECYCLE AND TERMINAL STATES
PATCH /api/orders/101/update_status/ (status: processing)
UPDATE orders_order SET status = processing WHERE id = 101
Status Updated
HTTP 200 OK (Order transitioned to PROCESSING)
PATCH /api/orders/101/update_status/ (status: shipped)
UPDATE orders_order SET status = shipped WHERE id = 101
Status Updated
HTTP 200 OK (Order transitioned to SHIPPED)
PATCH /api/orders/101/update_status/ (status: delivered)
UPDATE orders_order SET status = delivered WHERE id = 101
Status Updated (Terminal State Lock Engaged)
HTTP 200 OK (Order transitioned to DELIVERED)
Attempting illegal transition from Terminal State
PATCH /api/orders/101/update_status/ (status: processing)
FSM Validation: delivered is an immutable terminal state!
HTTP 400 Bad Request (Invalid status transition, delivered orders are immutable)